شرح مشروع قاعدة البيانات الوطنية للباحثين – من البداية إلى المرحلة الحالية

أولًا: خارطة الطريق الكاملة لبناء المشروع

هذه هي المراحل التي نمشي عليها لبناء المنصة من البداية حتى تصبح نظامًا متكاملًا وجاهزًا للاستخدام.

المسار الكامل لتطوير المنصة

1. المرحلة الأولى: تحليل النظام وتحديد المتطلبات

فهم فكرة المشروع، المشكلة التي يحلها، المستخدمين، الصلاحيات، الوظائف والمتطلبات.

2. المرحلة الثانية: التصميم والتحليل الهيكلي

Use Case، Activity Diagram، ERD، تصميم قاعدة البيانات، العلاقات بين الجداول وتدفق العمليات.

3. المرحلة الثالثة: تجهيز بيئة العمل وإنشاء المشروع

تثبيت الأدوات، إنشاء مشروعASP.NET Core ، تحديد التقنيات والمكتبات وربط قاعدة البيانات.

4. المرحلة الرابعة: بناء هيكل قاعدة البيانات(Entities)

إنشاء الكيانات التي تمثل جداول النظام وتحديد بيانات كل كيان.

5. المرحلة الخامسة: ضبط العلاقات والإعدادات(Configurations)

تحديد المفاتيح الأساسية والأجنبية، العلاقات، القيود والفهارس.

6. المرحلة السادسة: إنشاء قاعدة البيانات وتطبيقMigrations

تحويل تصميم قاعدة البيانات الموجود في الكود إلى جداول فعلية في.PostgreSQL

7. المرحلة السابعة: بناء نظام المستخدمين وتسجيل الدخول والصلاحيات

Register، Login، JWT، Roles، Permissions، حماية الحسابات والعمليات.

8. المرحلة الثامنة: بناء DTOs والتحقق من البيانات

تحديد البيانات التي يستقبلها النظام ويرسلها، والتحقق من صحتها قبل الحفظ.

9. المرحلة التاسعة: بناء) Services منطق النظام(

تنفيذ العمليات الفعلية الخاصة بالباحثين والجهات والمؤهلات والإنتاج العلمي وغيرها.

10. المرحلة العاشرة: بناء Controllers وAPI Endpoints

تجهيز نقاط الاتصال التي تستقبل طلبات المستخدمين وتوصلها إلى وظائف النظام.

11. المرحلة الحادية عشرة: تنفيذ وظائف المنصة الأساسية

ملفات الباحثين، التحقق والاعتماد، طلبات الباحثين، المؤسسات، الإنتاج العلمي، الفعاليات، الاستيراد وغيرها.

12. المرحلة الثانية عشرة: البحث المتقدم والتقارير ومؤشرات الأداءBI

البحث والتصفية والتحليل والإحصائيات ولوحات المعلومات والتصدير.

13. المرحلة الثالثة عشرة: بناء الواجهة الأماميةReact

تصميم صفحات المنصة وربطها بالخدمات البرمجية.

14. المرحلة الرابعة عشرة: التكامل بين الواجهة الأمامية والخلفية

جعل كل الأزرار والنماذج والصفحات تعمل فعليًا مع قاعدة البيانات.

15. المرحلة الخامسة عشرة: الاختبارات ومعالجة الأخطاء

اختبار الوظائف والصلاحيات والعلاقات والبيانات والأمان والأداء.

16. المرحلة السادسة عشرة: النشر والتشغيل والتوثيق

تجهيز النظام للاستضافة، نشره، إعداد النسخ الاحتياطية والتوثيق والتسليم.

هذه خارطة الطريق الكاملة، وقد نرجع أحيانًا إلى مرحلة سابقة لتعديل شيء أو تحسينه، وهذا طبيعي جدًا في تطوير الأنظمة.

ثانيا: ما الذي أنجزناه حتى الآن؟ وكيف أنجزناه؟

تم إنجاز الأساس البرمجي الأولي

.1إنشاء مشروعASP.NET Core

أول شيء بعد التحليل والتصميم كان الانتقال من المخططات إلى البرمجة.

فتحنا Visual Studio وأنشأنا مشروعًا جديدًا باستخدام.ASP.NET Core

طيب، ما معنىASP.NET Core ؟

ببساطة هو إطار عمل(Framework) ، يعني مجموعة أدوات وتجهيزات تساعدنا على بناء النظام الخلفي بدل ما نبدأ كل شيء من الصفر.

والنظام الخلفي هو الجزء الذي لا يراه المستخدم عادة، لكنه المسؤول عن تنفيذ العمليات.

مثلًا عندما يدخل باحث إلى المنصة ويحاول تسجيل بياناته، فالواجهة تعرض له نموذج التسجيل، لكن الجزء الخلفي هو الذي يستقبل البيانات ويفحصها ويحفظها.

اخترنا مشروعWeb API ؛ لأنه يسمح للواجهة الأمامية بالتواصل مع النظام الخلفي من خلال طلبات محددة.

ما معنىAPI ؟

هي واجهة اتصال بين برنامجين.

)تخيلوا أن React يتكلم معASP.NET Core ، فيرسل له طلبًا مثل: أعطني بيانات الباحث، فيستقبل ASP.NET الطلب ويجهز الإجابة ويرسلها له.(

بعد إنشاء المشروع ظهر لنا عدد من الملفات الأساسية، وأهمها:

الملف	وظيفته
Program.cs	نقطة تشغيل التطبيق وتجهيز الخدمات والإعدادات
appsettings.json	حفظ إعدادات التطبيق، مثل إعدادات الاتصال بقاعدة البيانات
NationalResearchers.csproj	تعريف المشروع وإصداره والمكتبات التي يحتاجها
Controllers	المجلد الذي سنضع فيه وحدات استقبال طلباتAPI
Data	المجلد المسؤول عن الاتصال بقاعدة البيانات وإعدادها
Entities	المجلد الذي نضع فيه الكيانات التي تمثل جداول قاعدة البيانات
DTOs	المجلد المخصص لنماذج البيانات التي تدخل وتخرج منAPI
Services	المجلد المخصص لمنطق العمليات البرمجية

إنشاء المجلدات لا يعني أننا أكملنا وظائفها؛ بعضها موجود لتجهيز مكان العمل للخطوات القادمة.

.2تثبيت المكتبات وربط قاعدة البيانات

بعد إنشاء المشروع، احتجنا إلى إضافة مكتبات تساعدنا على تنفيذ العمل.

ما معنى Package أو مكتبة؟

هي كود برمجي جاهز يوفر علينا كتابة وظائف من الصفر.

مثلًا بدل ما نكتب كل التعليمات اللازمة للتعامل مع قاعدة البيانات بأنفسنا، نستخدم مكتبة متخصصة.

أهم المكتبات التي ثبتناها:

لماذا استخدمناها؟	المكتبة
التعامل مع قاعدة البيانات عن طريق الكيانات	Microsoft.EntityFrameworkCore
أدوات التصميم اللازمة لعملياتEF Core	Microsoft.EntityFrameworkCore.Design
أدوات إدارة قاعدة البيانات والـMigrations	Microsoft.EntityFrameworkCore.Tools
تمكين EF Core من التعامل معPostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL
توفير Swagger لاستكشاف وتجربةAPI	Swashbuckle.AspNetCore
مكتبة أضيفت لمعالجة اعتماد أمني مرتبط بالتبعيات	System.Security.Cryptography.Pkcs

طيب، ما هوEntity Framework Core ؟

هذا من أهم المصطلحات التي لازم نفهمها.

هوORM ، واختصار.Object Relational Mapper

يعني أداة تربط عالم البرمجة بعالم قواعد البيانات.

بدل ما نكتب أوامر SQL لكل عملية بصورة مباشرة، نستطيع التعامل مع الكيانات المكتوبة بلغةC# ، و EF Coreيتولى تحويل العمليات إلى أوامر مناسبة لقاعدة البيانات.

مثال:

عندنا كيان اسمه Researcher يمثل الباحث.

عندما نطلب حفظ بيانات باحث جديد، يساعدنا EF Core على تحويل بيانات الكيان إلى سجل داخل جدول الباحثين في PostgreSQL.

طيب، ما هوPostgreSQL ؟

هو نظام إدارة قواعد بيانات علائقية، يعني يخزن البيانات في جداول مترابطة.

مثلًا:

- جدول المستخدمين.
- جدول الباحثين.
- جدول المؤسسات.
- جدول المؤهلات.

وكل جدول يحتوي على أعمدة وصفوف.

الصف يمثل سجلًا واحدًا، والعمود يمثل نوعًا معينًا من البيانات.

3.تجهيز الاتصال بين المشروع وقاعدة البيانات

بعد تثبيت المكتبات، احتجنا أن نخبر التطبيق كيف يتصل بقاعدة البيانات.

هنا عدلنا ملف .Program.cs

أضفنا إعدادًا يخبر المشروع باستخدامPostgreSQL ، ويقرأ بيانات الاتصال من .appsettings.json

يعني بدل ما يكتب التطبيق إعدادات الاتصال داخل الكود بشكل مباشر، يأخذها من ملف الإعدادات.

وهنا ظهر لنا مصطلح مهم:

Dependency Injection – DI

يعني حقن الاعتمادية.

الاسم يبدو صعبًا، لكن الفكرة سهلة.

تخيلوا أن عندنا موظفًا يحتاج جهاز كمبيوتر حتى ينفذ عمله. بدل ما يشتري الكمبيوتر بنفسه، المؤسسة توفره له. هذا تقريبًا DI ما يفعله؛ يوفر للكلاس الخدمة التي يحتاجها).

وفي مشروعنا استخدمنا:

AddDbContext

لتسجيل AppDbContextحتى يستطيع التطبيق استخدامه عند الحاجة.

كذلك جهزنا.Swagger

ما هوSwagger ؟

أداة تعرض لنا واجهات API وتسمح لنا بتجربة الطلبات أثناء التطوير.

يعني عندما نبدأ بناء API للباحثين، نستطيع تجربة الطلبات ومعرفة الاستجابات دون الحاجة إلى بناء واجهة React أولًا.

.4بناء – Entities الكيانات

تم إنشاء الكيانات

هذه من أهم الخطوات التي أنجزناها حتى الآن.

خلونا نفهم أولًا: لماذا نحتاجEntities ؟

ارجعوا إلى مخططات قاعدة البيانات التي عملناها في مرحلة التحليل.

كنا نحدد فيها الجداول والبيانات التي نحتاج تخزينها.

مثلًا قلنا إن النظام يحتاج جدولًا للباحثين، وجدولًا للمؤسسات، وجدولًا للمؤهلات.

الآن جاء دور تحويل هذا التصميم إلى كود.#C

وهنا نستخدم.Entities

ما معنىEntity ؟

هي كلاس (Class) يمثل كيانًا في النظام، ويُستخدم مع EF Core لتمثيل جدول في قاعدة البيانات.

طيب، ما هوClass ؟

هو قالب أو مخطط نحدد داخله البيانات والخصائص التي نحتاجها.

مثال من الحياة:

تخيلوا أن عندنا استمارة موحدة لتسجيل الباحثين، وكل باحث نعطيه استمارة تحتوي على الاسم، البريد الإلكتروني، الدرجة).
.(، وكل باحث نسجل بياناته يمثل سجلًا في الجدولClass العلمية وغيرها. هذا القالب هو الـ

مثلًا لدينا ملف:

Researcher.cs

يمثل الباحث، ويحتوي على خصائص تحدد البيانات التي نحتاج تخزينها عنه.

ويوجد ملف:

Organization.cs

يمثل المؤسسة أو الجهة الأكاديمية.

ويوجد:

Qualification.cs

يمثل المؤهل العلمي.

وهكذا.

ما الكيانات التي أنشأناها؟

في النسخة التي راجعناها من المشروع أنشأنا 39ملفEntity ، ويمكن تقسيمها حسب وظيفتها إلى مجموعات:

وظيفتها	أمثلة على الكيانات	المجموعة
إدارة الحسابات والأدوار والصلاحيات	User, Role, Permission, UserRole, RolePermission	المستخدمون والصلاحيات
بيانات الباحث وطلباته وروابطه الأكاديمية	Researcher, ResearcherRequest, ExternalAcademicLink	الباحثون وملفاتهم
المؤسسات وعلاقة الباحث بها	Organization, ResearcherOrganizationAffiliation	المؤسسات الأكاديمية
المؤهلات والتخصصات العلمية	Qualification, ResearcherQualification, ResearchSpecialty, ResearcherSpecialty	المؤهلات والتخصصات
اهتمامات الباحثين	ResearchInterest, ResearcherInterest	الاهتمامات البحثية
الإنتاج العلمي وتقاريره	ScientificProduction, ResearcherScientificProduction, ScientificProductionReport	الإنتاج العلمي
المشاركات والإعلانات والفعاليات	Participation, ResearcherParticipation, EventAnnouncement, ResearchSpecialtyAnnouncement	المشاركات والفعاليات
مراجعة البيانات وتقييمها وأدلة التحقق	WorkEvaluation, FormValidation, FieldReviewDetail, ValidationEvidence	التقييم والتحقق
استيراد البيانات ومتابعة السجلات المستوردة	ImportBatch, ImportedRecord	الاستيراد
تعريف المؤشرات وحفظ بياناتها ولقطاتها	IndicatorDefinition, IndicatorSnapshot, ResearcherExternalIndicator	المؤشرات والتحليل
الرموز وسجل العمليات والإشعارات	Token, AuditLog, Notification	المتابعة والتنبيهات

هذه الكيانات ليست مجرد أسماء؛ كل واحد منها يمثل جزءًا من وظائف المنصة التي حددناها في التحليل.

طيب، هل مجرد إنشاء Entities يعني أن قاعدة البيانات أصبحت جاهزة؟

لا.

نحن الآن جهزنا القوالب التي تصف الجداول، لكن ما زال علينا تحديد تفاصيل العلاقات والقيود بينها، وهنا تأتي الخطوة التالية.

5.بناء – AppDbContext نقطة الربط مع قاعدة البيانات

تم إنشاؤه وتجهيزه

بعد إنشاءEntities ، احتجنا إلى ملف يجمعها ويخبر EF Core بالجداول التي يتعامل معها.

وهنا أنشأنا:

AppDbContext.cs

ما هو هذا الملف؟

هو حلقة الوصل الأساسية بين الكيانات وقاعدة البيانات.

خلونا نبسطها:

تخيلوا أن عندنا أرشيفًا كبيرًا فيه عشرات الملفات، ونحتاج موظفًا مسؤولًا عن تنظيم الوصول إليها ومعرفة الملفات الموجودة فيه.

هذا تقريبًا دور.AppDbContext

داخله كتبنا DbSetلكل مجموعة من الكيانات.

ما معنىDbSet ؟

يعني أننا نخبر EF Core أن هذا الكيان يمثل مجموعة سجلات نتعامل معها كجدول.

مثلًا:

DbSet<Researcher>

يعني أن لدينا مجموعة الباحثين التي نتعامل معها في قاعدة البيانات.

وكتبنا أيضًا:

DbSet<User>

للمستخدمين، و <DbSet<Organizationللمؤسسات، وهكذا.

إذن AppDbContext أصبح يجمع الجداول التي نريد أن يتعامل معها النظام.

لكن بقيت نقطة مهمة جدًا: كيف يعرف EF Core العلاقات الدقيقة بين هذه الجداول؟

وهنا وصلنا إلى أحد أهم أجزاء العمل.

6.بناء – Configurations ضبط إعدادات الجداول والعلاقات

تمت مراجعة ملفات الإعدادات وإنشاؤها

أولًا، ما معنىConfiguration ؟

يعني إعدادات وتحديد قواعد معينة للكيان داخل قاعدة البيانات.

خلونا نفرق بين Entities وConfigurations:

Configurations	Entities
تحدد كيف سيُبنى الجدول وكيف يرتبط بالجداول الأخرى	تحدد ما هو الكيان والبيانات التي يحتوي عليها
تمثل قواعد تنظيم البيانات	تمثل وصف البيانات
مثلًا: الباحث يرتبط بحساب مستخدم واحد	مثلًا: الباحث لديه رقم واسم وبريد

)تخيلوا أننا نصمم منزلًا Entity :هو وصف الغرف الموجودة في المنزل، أما Configuration فهو الذي يحدد كيف ترتبط الغرف ببعضها وأين تكون الأبواب والمداخل.(

أولًا: ما هيPrimary Key ؟

.المفتاح الأساسي

.هو رقم أو قيمة تميز كل سجل عن غيره داخل الجدول

:مثال

:لدينا ثلاثة باحثين

ResearcherId	Name
1	أحمد
2	محمد
3	سارة

هنا ResearcherIdهو المفتاح الأساسي، وكل باحث له رقم لا يتكرر.

لماذا نحتاجه؟

حتى نستطيع الوصول إلى سجل معين وتمييزه عن بقية السجلات.

ثانيًا: ما هيForeign Key ؟

المفتاح الأجنبي.

هو حقل يستخدم لربط جدول بجدول آخر.

مثال:

عندنا جدول الباحثين وجدول المؤسسات.

كل باحث ينتسب إلى مؤسسة معينة.

بدل ما نكرر جميع بيانات المؤسسة داخل جدول الباحث، نستخدم رقم المؤسسة OrganizationIdلربط الباحث بالمؤسسة.

إذن المفتاح الأجنبي هو الذي يساعدنا على إنشاء العلاقات بين الجداول.

ثالثًا: ما أنواع العلاقات التي نحتاجها؟

مثال من مشروعنا	معناها	العلاقة
حساب المستخدم وملف الباحث	واحد إلى واحد	One-to-One
مؤسسة واحدة يمكن أن ترتبط بعدة باحثين	واحد إلى متعدد	One-to-Many
الباحث لديه عدة تخصصات والتخصص الواحد يمكن أن يرتبط بعدة باحثين	متعدد إلى متعدد	Many-to-Many

خلونا نفهم العلاقة الأخيرة أكثر.

الباحث الواحد يمكن أن يمتلك عدة تخصصات، والتخصص الواحد يمكن أن ينتمي إليه عدة باحثين.

كيف نمثل هذه العلاقة؟

ننشئ جدولًا وسيطًا اسمه:

ResearcherSpecialty

هذا الجدول يحتوي على رقم الباحث ورقم التخصص، وبالتالي يستطيع ربط الطرفين.

وهذا بالضبط ما فعلناه في عدد من العلاقات داخل المشروع.

رابعًا: ما معنىDeleteBehavior.Restrict ؟

هذا إعداد مهم استخدمناه في العلاقات.

معناه أننا نمنع حذف سجل مرتبط بسجلات أخرى إذا كان حذفُه سيؤدي إلى مشكلة في ترابط البيانات.

مثال:

لدينا جامعة مرتبطة بعدد من الباحثين.

لو حاولنا حذف الجامعة وهي ما زالت مرتبطة بباحثين، نريد أن يمنع النظام الحذف بدل أن يحذف البيانات المرتبطة تلقائيًا.

هذا يساعدنا على حماية البيانات من الحذف غير المقصود.

خامسًا: ما معنى Index وUnique ؟

هنا عندنا مصطلحان:

- Index:فهرس يساعد قاعدة البيانات على البحث عن البيانات والوصول إليها بشكل أسرع.
- Unique:قيد يمنع تكرار قيمة أو مجموعة قيم محددة.

مثال من مشروعنا:

في ResearcherSpecialtyConfigurationوضعنا قيدًا فريدًا على:

ResearcherId + ResearchSpecialtyId

لماذا؟

لأننا لا نريد أن نسجل نفس التخصص للباحث نفسه مرتين.

أي أن الباحث يستطيع امتلاك عدة تخصصات، لكن لا يستطيع تسجيل التخصص نفسه مرتين في جدول الربط.

ما الملفات التي أنجزناها فيConfigurations ؟

في النسخة التي راجعناها يوجد 11 ملف إعدادات:

وظيفته	الملف
ضبط العلاقات المباشرة لمجموعة من الكيانات	DirectRelationshipsConfiguration
ضبط علاقة المؤسسة بالمؤسسة الأم	OrganizationConfiguration
ربط المستخدمين بالأدوار	UserRoleConfiguration
ربط الأدوار بالصلاحيات	RolePermissionConfiguration
ربط الباحثين بالتخصصات	ResearcherSpecialtyConfiguration
ربط الباحثين بالاهتمامات البحثية	ResearcherInterestConfiguration
ربط الباحثين بالمؤهلات	ResearcherQualificationConfiguration
ربط الباحثين بالمشاركات	ResearcherParticipationConfiguration
ربط الباحثين بالإنتاج العلمي	ResearcherScientificProductionConfiguration
ربط الباحثين بالمؤسسات	ResearcherOrganizationAffiliationConfiguration
ربط التخصصات بالإعلانات	ResearchSpecialtyAnnouncementConfiguration

لاحظوا أن DirectRelationshipsConfigurationمختلف قليلًا؛ لأنه يحتوي على إعدادات عدة كيانات داخل ملف واحد بدل تخصيص ملف منفصل لكل كيان.

كيف جعلنا EF Core يقرأ هذه الإعدادات؟

رجعنا إلى AppDbContext، وداخل الدالة:

OnModelCreating

استخدمنا:

ApplyConfigurationsFromAssembly

معناها أن EF Core يبحث عن ملفات الإعدادات الموجودة في المشروع ويطبقها تلقائيًا.

بدل ما نذهب لكل ملف ونربطه يدويًا داخل.AppDbContext

هكذا أصبح عندنا تنظيم واضح Entities :تصف الجداول، و Configurationsتضبط تفاصيلها وعلاقاتها، و AppDbContextيجمعها.

.7مراجعة الكود وتصحيح العلاقات

بعد كتابة الكيانات والإعدادات، لم نكتفِ بإنشاء الملفات فقط.

راجعنا العلاقات بين الكيانات وقارنّاها بما اتفقنا عليه في تصميم قاعدة البيانات.

مثلًا:

- راجعنا علاقة المؤسسة بالمؤسسة الأم.
- راجعنا علاقات الباحث بالتخصصات والمؤهلات والاهتمامات.
- تأكدنا من استخدام الجداول الوسيطة في علاقات.Many-to-Many
- راجعنا المفاتيح الأجنبية وسلوك الحذف.
- أضفنا علاقة الباحث بحسابه في الكيان .Researcher
- عدّلنا ExternalAcademicLinkليكون ارتباطه بالباحث عن طريق ResearcherIdبدل .AccountId
- راجعنا علاقة الإعلانات بمنشئيها ومعدّليها.

لماذا هذه المراجعة مهمة؟

لأن أي خطأ في العلاقات قد يسبب مشكلات لاحقًا عند حفظ البيانات أو استخراجها أو حذفها.

يعني لو صممنا العلاقة بشكل غير صحيح من البداية، قد نضطر لتعديل عدد كبير من الملفات عندما نبدأ تنفيذ الوظائف.

رابعًا: أين وصلنا بالضبط؟ وما الذي لم ننجزه بعد؟

خلونا نكون واضحين حتى لا نخلط بين الملفات التي أنشأناها والوظائف التي أصبحت تعمل.

الجزء	وضعه في النسخة التي راجعناها
إنشاء مشروعASP.NET Core 10	منجز
تثبيت المكتبات الأساسية	منجز
تجهيز الاتصال بـ PostgreSQL في الإعدادات	منجز برمجيًا
إنشاءEntities	منجز

منجز	إنشاءAppDbContext
منجز	إنشاءConfigurations
منجزة في الملفات المراجعة	مراجعة العلاقات الأساسية
لم تُبنَ بعد	Controllersالخاصة بوظائف المنصة
لم تُبنَ بعد	DTOsالخاصة بالعمليات
لم يُنفذ بعد	Servicesومنطق العمليات
لم يُنجز بعد في النسخة التي راجعناها	إنشاء Migrations وتطبيقها
لم تُنفذ بعد	تسجيل الدخول و JWTوالصلاحيات الفعلية
لم تُنفذ بعد	وظائف المنصة
لم يُبنَ بعد في هذه النسخة	React
مراحل لاحقة	BIوالتقارير والاختبارات والنشر

ملاحظة مهمة: يوجد في مجلد Controllers ملف تجريبي اسمه WeatherForecastController.cs، لكنه ليس Controllerخاصًا بوظائف منصتنا.

كذلك وجود مجلدي DTOs و Servicesلا يعني أن العمليات البرمجية قد بُنيت فيهما؛ هما مجهزان لاستقبال العمل القادم.

إذن ماذا أنجزنا فعليًا؟

أنجزنا الأساس الذي سيُبنى عليه النظام:

1. جهزنا بيئة المشروع.
2. اخترنا التقنيات والمكتبات.
3. جهزنا الاتصال بقاعدة البيانات.
4. حولنا تصميم الجداول إلى.Entities
5. ضبطنا العلاقات والقيود من خلال.Configurations
6. جمعنا الكيانات داخل.AppDbContext
7. نظمنا المشروع ورفعناه على.GitHub

وهذا أساس مهم جدًا، لكنه لا يعني أن المنصة أصبحت تعمل بكامل وظائفها؛ لأننا ما زلنا بحاجة إلى تنفيذ العمليات والخدمات والواجهات.

خامسًا: ما المرحلة التالية التي سنبدأ بها؟

المرحلة القادمة

إنشاء قاعدة البيانات فعليًا باستخدامMigrations

بعد أن جهزنا الكيانات والعلاقات والإعدادات، أصبح لدينا وصف برمجي لشكل قاعدة البيانات.

لكن هل تحولت هذه الكيانات إلى جداول فعلية داخلPostgreSQL ؟

ليس بعد.

وهنا يأتي دور.Migrations

ما معنىMigration ؟

هي آلية في Entity Framework Core تسمح لنا بتحويل التغييرات التي أجريناها على تصميم قاعدة البيانات في الكود إلى تعليمات لإنشاء الجداول وتعديلها في قاعدة البيانات الفعلية.

خلونا نوضحها بمثال:

تخيلوا أننا رسمنا مخططًا لمبنى وحددنا الغرف والأبواب والعلاقات بينها، لكن المبنى لم يُبنَ بعد.

- Entitiesهي وصف الغرف والبيانات.
- Configurationsتحدد تفاصيل البناء والعلاقات.
- Migrationهي خطة التنفيذ التي تحول التصميم إلى قاعدة بيانات فعلية.
- PostgreSQLهي المكان الذي سيُبنى فيه هذا الهيكل ويُحفظ فيه.

ماذا سنفعل في المرحلة القادمة؟

1. نتأكد من سلامة المشروع وعدم وجود أخطاء برمجية.
2. نتأكد من إعدادات الاتصال بقاعدة البيانات.
3. ننشئ أول.Migration
4. نراجع التغييرات التي ستُطبق على قاعدة البيانات.
5. ننفذ تحديث قاعدة البيانات باستخدام.EF Core
6. نتأكد من إنشاء الجداول والعلاقات بصورة صحيحة.

وبعد التأكد من نجاح هذه المرحلة، ننتقل إلى بناء نظام المستخدمين وتسجيل الدخول والصلاحيات، ثم نبدأ بتنفيذ وظائف المنصة.